

Core Language Working Group "ready" Issues for the November, 2017 (Albuquerque) meeting

Section references in this document reflect the section numbering of document [WG21 N4659](#).

2342. Reference `reinterpret_cast` and pointer-interconvertibility

Section: 8.2.10 [`expr.reinterpret.cast`] **Status:** ready **Submitter:** Richard Smith **Date:** 2017-04-07

The changes from document P0137 make it clear that this is valid:

```
struct A { int n; } a;  
int *p = reinterpret_cast<int*>(&a); // ok, a and a.n are pointer-interconvertible  
int m = *p;                        // ok, p points to a.n
```

but the handling for that case does not extend to this one:

```
int &r = reinterpret_cast<int&>(a);  
int n = r;
```

The relevant rule is 8.2.10 [`expr.reinterpret.cast`] paragraph 11:

A glvalue expression of type T1 can be cast to the type “reference to T2” if an expression of type “pointer to T1” can be explicitly converted to the type “pointer to T2” using a `reinterpret_cast`. The result refers to the same object as the source glvalue, but with the specified type. [Note: That is, for lvalues, a reference cast `reinterpret_cast<T&>(x)` has the same effect as the conversion `*reinterpret_cast<T*>(&x)` with the built-in `&` and `*` operators (and similarly for `reinterpret_cast<T&&>(x)`). —end note]

Note that the normative rule and the note specify different rules: under the rule described in the note, the result would be a reference to the object `a.n`. According to the normative rule, however, we get a reference to the object `a` with type `n`.

Proposed resolution (July, 2017):

Change 8.2.10 [`expr.reinterpret.cast`] paragraph 11 as follows:

A glvalue expression of type T1, **designating an object x**, can be cast to the type “reference to T2” if an expression of type “pointer to T1” can be explicitly converted to the type “pointer to T2” using a `reinterpret_cast`. The result **refers to the same object as the source glvalue, but with the specified type is that of `*reinterpret_cast<T2 *>(p)` where p is a pointer to x of type “pointer to T1”.** [Note: That is, for lvalues, a reference cast `reinterpret_cast<T&>(x)` has the same effect as the

~~conversion `*reinterpret_cast<T*>(&x)` with the built-in `&` and `*` operators (and similarly for `reinterpret_cast<T&&>(x)`). —end note~~ No temporary is created, no copy is made, and constructors (15.1 [class.ctor]) or conversion functions (15.3 [class.conv]) are not called. [*Footnote:* This is sometimes referred to as a *type pun* **when the result refers to the same object as the source glvalue**. —end footnote]